

Autonomous Module Operations Guide

Paper vs. live trading with IBKR, explained once and precisely. How every trade — Intelligence, Autonomous, or Manual — flows through one ticket pipeline into orders, positions, P&L, and reporting metrics, with paper and live modes always separable. Includes the deployment confirmation checklist with completion notes, the troubleshooting manual, and the accuracy questionnaire.

PAPER / LIVE MODE LINEAGE

ONE TICKET PIPELINE

DEPLOYMENT CHECKLIST

TROUBLESHOOTING

ACCURACY QUESTIONNAIRE

Contents

1 · Paper mode vs. live mode — the one-paragraph answer	3
2 · The golden rule: everything is a ticket (data lineage)	4
3 · Mode lineage — how paper/live is marked everywhere	6
4 · Where every number comes from (metrics dictionary)	7
5 · Deployment confirmation checklist	8
6 · Troubleshooting manual	13
7 · Accuracy questionnaire	15
Appendix A · Verification SQL & endpoint smoke list	17
Appendix B · The mode-lineage change (what shipped, exact diffs)	19

1 Paper mode vs. live mode — the one-paragraph answer

Paper mode means the platform runs the complete trading pipeline — scanning, state machines, risk gates, tickets, confirmations, fills, positions, trailing stops, P&L — but the "broker" on the ticket is `PAPER` : a simulated execution venue inside our own server. Fills are computed, not received; the account is a fixed paper equity of **\$100,000** used for position sizing; no money exists and nothing leaves the building. **Live mode** means the ticket's broker is a real connected broker account — today `IBKR` (Interactive Brokers, via the TWS/Gateway over the Model Context Protocol). The identical pipeline runs, but the order is transmitted to IBKR, the fill comes back from the exchange, the position is real, and the P&L is real money. **The engine does not have two code paths. It has one pipeline with a broker field on the ticket — and that field is the mode.**

1.1 Side by side

	Paper mode	Live mode (IBKR)
Ticket broker field	<code>PAPER</code>	<code>IBKR</code> (or the connected broker's name)
Where the order goes	Nowhere — simulated inside our server	Transmitted to IBKR TWS/Gateway via MCP
Fill price	Computed (stop/market simulation)	Real exchange fill from the broker ACK
Account equity for sizing	Fixed \$100,000 paper equity	Real account equity from the broker
Positions & P&L	Real rows in the same tables — flagged <code>PAPER</code>	Real rows in the same tables — flagged with the broker name
Disaster stop (resting broker-side)	Not placed (<code>disasterOrderId</code> stays null)	Placed at IBKR as a resting catastrophe stop
Risk gates, governance, CONFIRM string, trailing engine	Identical in both modes — that is the point of the design	

1.2 Why the modes can never silently mix

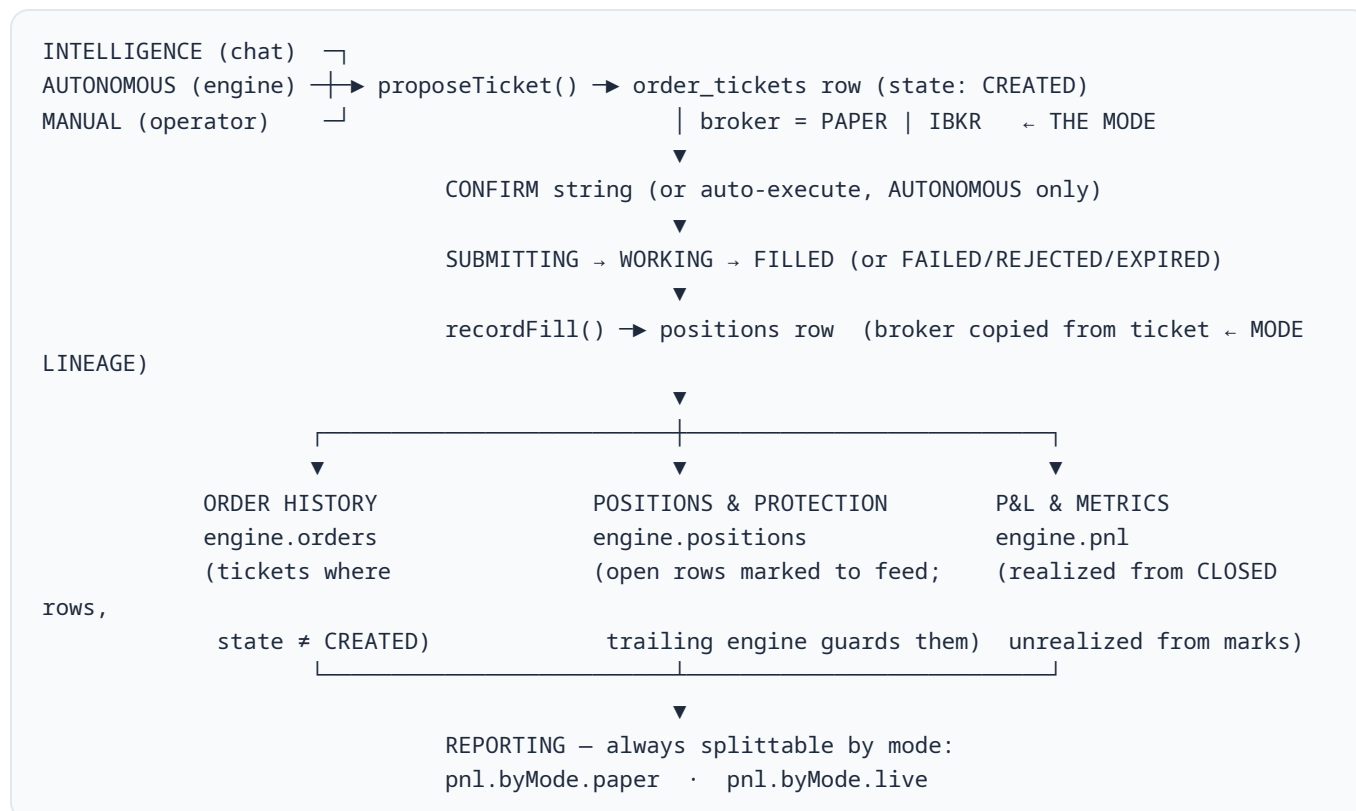
Every position inherits its mode at fill time from the ticket that created it — copied onto the position row itself (`positions.broker`). From that moment the mode is a fact of the data, not a UI setting: order history shows it, the positions table shows it, and the P&L engine splits on it. A paper position can never be marked as live and a live position can never be averaged into a paper one, because the mode travels with the row, not with a toggle.

CURRENT DEFAULT

Today the Intelligence chat proposes with `broker = PAPER` , and the autonomous engine proposes paper tickets until a live broker account is bound. If a real broker call degrades mid-flight, the system falls back to paper *and says so on the ticket* ("paper fallback" in the confirmation message). Nothing ever silently pretends to be live.

2 The golden rule: everything is a ticket (data lineage)

There is exactly one way for any trade to exist in Requi — whether it came from the Intelligence chat, the Autonomous engine, or a human clicking manual order: **it becomes an order ticket, and only a ticket may reach a broker.** Once you know the lineage, every screen and every number in the product is just a view over the same chain:



2.1 The five facts that make the lineage trustworthy

1. **One writer per stage.** Only `proposeTicket()` creates tickets; only the confirmation/auto-execute path submits them; only `recordFill()` opens positions. Three functions, one chain — there is no side door.
2. **The position remembers its ticket.** `positions.sourceTicketId` points back to the ticket that opened it; `positions.broker` freezes the mode at fill time.
3. **Scale-outs keep the parent's mode.** When a T1/T2 scale-out closes part of a position, the closed child row carries the *original position's* broker — never the sell ticket's.
4. **Order history is tickets, not a separate log.** The Orders panel is literally the ticket table minus unsubmitted drafts — so "did the order connect?" is answered by one row's state field.
5. **P&L is computed from positions, never from tickets.** Realized P&L comes only from CLOSED position rows; unrealized comes only from open rows marked to the live feed. Tickets tell you *what was attempted*; positions tell you *what is true*.

HOW TO AUDIT ANY TRADE IN 30 SECONDS

Find the ticket in Orders → note its state and broker badge → find the position it opened (`sourceTicketId`) → its broker badge matches the ticket's → its realized/unrealized P&L appears in the matching mode bucket of the P&L panel. If any link is missing, §6 (Troubleshooting) has the exact failure table.

3 Mode lineage — how paper/live is marked everywhere

This release (snapshot `4896a72`) makes the mode visible at every layer of the stack. The rule the whole system now obeys: **a number on any screen must be answerable to one mode or be an explicit, labeled combination of both.**

3.1 The lineage chain

Layer	Where the mode lives	How you see it
Ticket (order_tickets)	<code>broker</code> column — <code>PAPER</code> default, live broker name when bound	PAPER / LIVE badge in Orders, next to the ticket id
Position (positions)	<code>broker</code> column — copied from the source ticket at fill time (<i>new column, this release</i>)	PAPER / LIVE badge in Positions, next to the symbol
P&L summary	<code>byMode.paper</code> / <code>byMode.live</code> — realized, unrealized, open, closed, W/L per mode (<i>new, this release</i>)	Two mode cards under the P&L strip: sky = paper, teal = live
Combined totals	Unchanged, back-compatible	The four original P&L cards = paper + live combined, labeled as such

3.2 What the mode badges mean operationally

- **PAPER** — simulated fill, \$100k paper equity sizing, no broker-side disaster stop, no money at risk. Used for validation, demos, UAT, and the engine's default until a live account binds.
- **LIVE** — real broker execution, real equity, resting disaster stop at IBKR, real P&L. Every protection (governance, gates, CONFIRM, trailing) is identical.

3.3 The three mode questions you can now answer instantly

1. **"Is this trade real?"** → look at the badge on the ticket or position. Badge is the mode; the mode is the broker column; the column is set once at fill time and never edited.
2. **"What did we make on paper vs live?"** → the two mode cards under P&L. They never mix.
3. **"Did anything trade live that shouldn't have?"** → `SELECT * FROM positions WHERE broker != 'PAPER'` — if the answer surprises you, §6.7 is the incident path.

PRE-EXISTING ROWS

Positions created before this release have no broker value on the row; the guarded migration backfills them to `PAPER` — which is factually correct for every fill this platform has produced to date (no live account has been bound). If a live account was bound before this release, update those rows manually using the ticket lineage (`sourceTicketId` → `order_tickets.broker`) — SQL in Appendix A.4.

4 Where every number comes from (metrics dictionary)

When a metric "doesn't look right", it is always one of these definitions being misunderstood. This table is the contract — if a screen ever disagrees with it, the screen is wrong and this section is the fix target.

Metric	Definition (the contract)	Source
Realized P&L	Σ realizedPnl over CLOSED position rows only. A trade is not real P&L until it is closed.	positions, status=CLOSED
Unrealized P&L	Σ (live mark – avgEntry) × qty over OPEN rows <i>that have a mark</i> . Rows without a feed are excluded and counted, never estimated.	positions OPEN × feed last price
Unmarked positions	Open rows with no tracked feed. The P&L note names the count; the fix is tracking the symbol, not editing the number.	positions × marketDataService
Win / Loss	CLOSED rows with realizedPnl > 0 / ≤ 0. Breakeven exits count as losses — deliberate, conservative.	positions
Order history row	A ticket whose state ≠ CREATED. States: SUBMITTING, WORKING, FILLED, FAILED, REJECTED, EXPIRED.	order_tickets
Fill price	averageFillPrice on the ticket; paper = simulated, live = broker-reported. The position's avgEntry is the VWAP of all its fills.	order_tickets → positions
Mode of any row	The broker column. PAPER = simulated; anything else = the named live broker.	positions.broker / order_tickets.broker
Paper P&L / Live P&L	The same metrics computed over rows of one mode only. Combined cards = both modes, labeled.	pnl.byMode
Position risk (heat)	qty × (avgEntry – protective stop); unprotected = full notional. Used by the 10% heat gate.	positions + stops
Paper equity	Fixed \$100,000 — sizing base in paper mode until a live account binds. Not a P&L input.	engine constant

THE THREE CLASSIC "WRONG METRICS" BUGS — AND WHY THEY CAN'T HAPPEN HERE

1. Mixing attempted with executed — impossible: metrics read positions, attempts read tickets, different tables. **2. Mixing paper with live** — impossible as of this release: every row carries its mode and the split is structural. **3. Inventing marks** — impossible: unmarked rows are excluded and disclosed, never estimated (§5 of the Strategy Specification). If a metric ever seems wrong, first check *which mode bucket you're looking at* and *whether the position is marked* — those two checks resolve 95% of "wrong number" reports.

5 Deployment confirmation checklist

Work top to bottom against the deployment guide. Every item has a confirmation line — date and initial it — and a notes line for *how* it was completed (command output, screenshot ref, observation). An item without a note is an item not done.

☐ **A1 — Version snapshot noted** — record the previous snapshot id (current: 4896a72) as the rollback point in the deploy ticket

Completed (date / initials): _____

How it was completed / notes: _____

☐ **A2 — Build succeeds clean** — npm run build completes with dist/boot.js + dist/public; no type errors

Completed (date / initials): _____

How it was completed / notes: _____

☐ **A3 — No new env vars required** — this release adds none — confirm GOVERNANCE_VAULT_KEY and existing vars already set

Completed (date / initials): _____

How it was completed / notes: _____

☐ **B1 — Restart production** — NODE_ENV=production node dist/boot.js on port 3000

Completed (date / initials): _____

How it was completed / notes: _____

☐ **B2 — Boot log clean** — expect [db] schema ensured (16 tables) → Server running on http://localhost:3000/

Completed (date / initials): _____

How it was completed / notes: _____

☐ **B3 — Schema ALTERs applied** — DESCRIBE positions shows broker VARCHAR(24) NOT NULL DEFAULT 'PAPER' after sourceTicketId; DESCRIBE users shows autoUniverse + autoUniverseMax

Completed (date / initials): _____

How it was completed / notes: _____

☐ **B4 — Prior rows backfilled** — `SELECT COUNT(*) FROM positions WHERE broker IS NULL → 0`; all pre-existing rows read PAPER

Completed (date / initials): _____

How it was completed / notes: _____

☐ **C1 — engine.pnl** — 401 signed out; signed in → payload contains `byMode.paper` and `byMode.live` blocks

Completed (date / initials): _____

How it was completed / notes: _____

☐ **C2 — engine.orders** — each row carries a broker field

Completed (date / initials): _____

How it was completed / notes: _____

☐ **C3 — engine.positions** — each row carries a broker field

Completed (date / initials): _____

How it was completed / notes: _____

☐ **C4 — engine.strategyStats** — 401 signed out → confirms v2.5 strategy registry present (100 strategies, 2 eligible)

Completed (date / initials): _____

How it was completed / notes: _____

☐ **C5 — engine.universe** — 401 signed out → confirms auto-universe engine present

Completed (date / initials): _____

How it was completed / notes: _____

☐ **D1 — Orders badges** — every order row shows a PAPER (sky) or LIVE (teal) badge beside the ticket id

Completed (date / initials): _____

How it was completed / notes: _____

☐ **D2 — Positions badges** — every open position shows the mode badge beside the symbol

Completed (date / initials): _____

How it was completed / notes: _____

☐ **D3 — Mode split cards** — two cards under the P&L strip: Paper mode (sky) and Live mode (teal), each with realized, unrealized, open/closed, W/L

Completed (date / initials): _____

How it was completed / notes: _____

☐ **D4 — Combined cards labeled** — the four original P&L cards are understood as paper + live combined

Completed (date / initials): _____

How it was completed / notes: _____

☐ **D5 — Universe panel renders** — Autonomous page shows the auto-universe engine card + 100-strategy registry

Completed (date / initials): _____

How it was completed / notes: _____

☐ **E1 — Paper trade end-to-end** — propose via Intelligence chat (PAPER) → CONFIRM ORDER [id] → FILLED → position opens with PAPER badge

Completed (date / initials): _____

How it was completed / notes: _____

☐ **E2 — Lineage audit** — position.sourceTicketId = the ticket id; positions.broker = order_tickets.broker = PAPER

Completed (date / initials): _____

How it was completed / notes: _____

☐ **E3 — Metrics reflect mode** — the paper mode card's open count increments; live card unchanged; combined card increments

Completed (date / initials): _____

How it was completed / notes: _____

☐ **E4 — Close the trade** — flatten → realized P&L appears in the PAPER card only; win/loss counts update

Completed (date / initials): _____

How it was completed / notes: _____

☐ **E5 — Order history connects** — the closing SELL ticket appears in Orders with PAPER badge and FILLED state

Completed (date / initials): _____

How it was completed / notes: _____

☐ **F1 — Heartbeat ticking** — during market hours a monitor's timerCount advances with no UI clicks (60s cycle)

Completed (date / initials): _____

How it was completed / notes: _____

☐ **F2 — Trailing loop running** — System Check shows the 5s protective cycle LIVE/ARMED; disasterOrderId is null on paper positions

Completed (date / initials): _____

How it was completed / notes: _____

☐ **F3 — Governance verified** — boot shows governance ready; a tampered/unsigned package would halt — do not test on production

Completed (date / initials): _____

How it was completed / notes: _____

☐ **F4 — Market-hours stand-down** — outside 09:30–16:00 ET the heartbeat sleeps; screens show MARKET CLOSED, not errors

Completed (date / initials): _____

How it was completed / notes: _____

☐ **G1 — Questionnaire complete** — Section 7 answered in full; every answer matches its expected value or has an open ticket

Completed (date / initials): _____

How it was completed / notes: _____

☐ **G2 — Rollback rehearsed (verbally)** — the deployer can state the rollback path without looking it up (snapshot id + one command)

Completed (date / initials): _____

How it was completed / notes: _____

☐ **G3 — Deploy record filed** — this checklist, completed, attached to the deploy ticket

Completed (date / initials): _____

How it was completed / notes: _____

6 Troubleshooting manual

Symptom → cause → fix. Every entry names the check that proves the diagnosis before the fix is applied.

Symptom	Most likely cause	Prove it, then fix it
6.1 Screen looks "stuck"	Market closed (heartbeat stands down) — or pre-v2.5 build (no heartbeat at all)	Check the MARKET OPEN/CLOSED badge. Closed = correct behavior. Open + frozen: confirm the build is ≥ v2.5 (engine.universe must 401, not 404) and restart.
6.2 "No bars" on a symbol	Gateway down, no market-data subscription, or symbol not tracked	Feed row shows the exact error string. Track the symbol; check IBKR Gateway login; verify the symbol's subscription tier. Never fabricate bars — the engine refuses to.
6.3 Monitor not proposing	A gate is doing its job: OCO per-day, portfolio heat, correlation, timer not met, or chase limit	Read the monitor's detail string — it names the gate verbatim ("blocked by PORTFOLIO HEAT gate...", "already proposed for this symbol today..."). The reason IS the diagnosis.
6.4 Ticket stuck awaiting confirmation	CONFIRM string mismatch, or ticket expired (5:00 window)	String must be exact: CONFIRM ORDER [TICKET_ID]. Governance validates it against the signed package; STALE_OR_MISMATCHED_TICKET means retype exactly or re-propose.
6.5 P&L looks wrong	Wrong mode bucket, or unmarked positions	Check the two mode cards vs combined. Then the "unmarked" note — unrealized excludes positions with no feed by design (§4). Track the symbol; do not edit numbers.
6.6 Paper/live numbers mixed	Pre-4896a72 build, or rows predating the broker column	positions must have broker NOT NULL DEFAULT 'PAPER' (B3). Old rows backfill to PAPER automatically; if any pre-release row was actually live, fix via Appendix A.4.
6.7 Something traded live unexpectedly	Incident — stop the line	SELECT * FROM positions WHERE broker != 'PAPER'. If real: operator-kill the strategy, flatten via the position's flatten control, preserve the ticket chain (sourceTicketId) for forensics, then call the owner before any cleanup.
6.8 Auto-universe not scanning	Not opted in, market closed, or roster read failed (fail-closed)	Enable auto-universe in the UI; check MARKET badge; engine.universe → preference.enabled. Fail-closed roster means a DB issue — check connection, not the engine.
6.9 404 on a new endpoint	Stale deployment	401 = route exists, auth blocked (good). 404 = old build. Redeploy; confirm dist/boot.js timestamp.

6.10 Schema column missing after deploy	ensureSchema guard skipped or errored	Boot log must show "[db] schema ensured". Re-run boot; the ALTERs are idempotent. Apply manually from Appendix A.3 only if boot cannot run.
6.11 Trailing not firing	Market closed, loop down, or position is MANUAL override	System Check → trailing LIVE/ARMED/DOWN. Check overrideMode on the row — MANUAL means the human owns the exit by design. Paper positions never have disaster stops (disasterOrderId null is correct).
6.12 "GOVERNANCE_UNAVAILABLE" anywhere	No signed active package, or integrity failure	Do not bypass. Boot runs ensureGovernanceReady (genesis compile on first boot). Integrity failure = the vault/package was touched — treat as §6.7-class incident.

THE TWO RULES OF REQUI TROUBLESHOOTING

1. Read the reason string first. The engine narrates every refusal — gates, monitors, tickets all carry a verbatim reason. The message is the diagnosis; don't guess around it. **2. Never "fix" by editing data.** Numbers are derived from the ticket → fill → position chain. If the chain is wrong, fix the chain; if the number is wrong, find which link disagrees. Editing a derived number destroys the audit trail and converts a bug into a lie.

7 Accuracy questionnaire

Answer after the checklist. Each question has its expected answer printed beneath — if yours differs, that is a finding: open a ticket, do not proceed to sign-off. Space is left for your written answer.

1. What single field determines whether any trade is paper or live?

Your answer: _____

Expected: the broker column on the ticket (PAPER | IBKR), inherited by the position at fill time.

2. Where does the \$100,000 figure come from, and what is it used for?

Your answer: _____

Expected: the paper-equity engine constant — position sizing in paper mode only; never a P&L input.

3. Name the three functions that are the only writers in the trade chain.

Your answer: _____

Expected: proposeTicket() creates, the confirm/auto-execute path submits, recordFill() opens positions.

4. A scale-out closes half a position. Which broker value does the closed child row get, and why?

Your answer: _____

Expected: the ORIGINAL position's broker — the mode belongs to the position's birth, not to the closing ticket.

5. Why does the Orders panel exclude CREATED tickets?

Your answer: _____

Expected: CREATED = unconfirmed draft, never submitted; order history shows what actually reached a broker path.

6. Realized P&L is computed from which table and which rows?

Your answer: _____

Expected: positions where status = CLOSED — never from tickets.

7. An open position has no feed. What happens to its unrealized P&L?

Your answer: _____

Expected: excluded from unrealized totals and counted in "unmarked" — never estimated.

8. Where on the P&L panel do you read paper-only results? Live-only?

Your answer: _____

Expected: the sky Paper mode card and the teal Live mode card under the main strip (byMode.paper / byMode.live).

9. What does a PAPER badge on a position guarantee about its disaster stop?

Your answer: _____

Expected: none resting broker-side — disasterOrderId is null on paper; the in-server trailing engine still protects it.

10. Which two badges must match for a lineage audit to pass?

Your answer: _____

Expected: the ticket's broker badge and the position's broker badge (via sourceTicketId).

11. One of your checks returns 404 instead of 401. What does that mean and what do you do?

Your answer: _____

Expected: stale deployment — redeploy; 401 means the route exists and auth blocked correctly.

12. What are the heartbeat's three steps, and its frequency?

Your answer: _____

Expected: refresh all feeds → evaluate every monitor → auto-universe cycles; every 60s, market hours only.

13. The trailing engine's cycle time, and what it owns exclusively?

Your answer: _____

Expected: 5 seconds, market hours; it owns every protective exit — no strategy may move a stop but tighter.

14. Name the four states a ticket can end in without filling.

Your answer: _____

Expected: FAILED, REJECTED, EXPIRED (and CREATED drafts that never submitted).

15. Why can the Intelligence chat never auto-execute, even if a user asks nicely?

Your answer: _____

Expected: auto-execute is scoped to AUTONOMOUS-origin tickets only; INTELLIGENCE and MANUAL always require the CONFIRM string — code-enforced.

16. Which ALTER must exist for mode lineage, and what is its default?

Your answer: _____

Expected: positions.broker VARCHAR(24) NOT NULL DEFAULT 'PAPER' — applied by the guarded ensure-schema migration.

17. A monitor's detail says "blocked by CORRELATION gate — cannot verify vs open position XYZ". Is this a bug?

Your answer: _____

Expected: no — \$5 never-estimate rule; unverifiable correlation data rejects the proposal by design.

18. What two pieces of evidence must accompany every checked checklist item?

Your answer: _____

Expected: date + initials on the confirmation line, and a written "how it was completed" note.

19. When is editing a P&L number directly in the database acceptable?

Your answer: _____

Expected: never. Fix the chain (ticket → fill → position); derived numbers are recomputed, not edited.

20. State the rollback path for this release.

Your answer: _____

Expected: restore snapshot 4896a72 (or the prior one recorded at A1) via the version manager — one step.

SCORING

20/20 matching expected answers → sign off. Any mismatch → the mismatch is a defect report: quote the question number, your answer, and the observed evidence. Do not average, do not proceed.

A Appendix A · Verification SQL & endpoint smoke list

A.1 Mode-lineage audit (the 30-second trade audit)

```
-- Trace one position back to its ticket and confirm the modes match
SELECT p.id, p.symbol, p.broker AS position_mode,
       p.sourceTicketId, t.broker AS ticket_mode, t.state, t.averageFillPrice
FROM positions p
LEFT JOIN order_tickets t ON t.ticketId = p.sourceTicketId
ORDER BY p.id DESC
LIMIT 20;
-- PASS rule: position_mode = ticket_mode on every row.
```

A.2 Mode-separated P&L (server computes this — use it to cross-check)

```
SELECT broker AS mode, status,
       COUNT(*) AS rows_n,
       ROUND(SUM(COALESCE(realizedPnl, 0)), 2) AS realized
FROM positions
GROUP BY broker, status
ORDER BY broker, status;
-- Compare against the Paper/Live cards on the P&L panel. Must agree.
```

A.3 Schema verification

```
DESCRIBE positions; -- expect: broker VARCHAR(24) NOT NULL DEFAULT 'PAPER' AFTER sourceTicketId
DESCRIBE users;     -- expect: autoUniverse TINYINT(1) DEFAULT 0, autoUniverseMax INT DEFAULT 5
SELECT COUNT(*) AS missing_mode FROM positions WHERE broker IS NULL; -- expect 0
```

A.4 Manual backfill (only if a pre-release row was genuinely live)

```
-- Fix one position's mode from its ticket lineage (incident-path only, log the change)
UPDATE positions p
JOIN order_tickets t ON t.ticketId = p.sourceTicketId
SET p.broker = t.broker
WHERE p.id = <position_id> AND t.broker != 'PAPER';
```

A.5 Unexpected-live check (run after every deploy until live binding is intentional)

```
SELECT * FROM positions WHERE broker != 'PAPER';
-- Expect: zero rows until a live IBKR account is deliberately bound.
-- Any row = incident path $6.7.
```

A.6 Endpoint smoke list (signed out → 401; signed in → payload shape)

Endpoint	Signed-in payload must contain
engine.pnl	byMode.paper + byMode.live , each with realizedTotal, unrealizedTotal, openCount, closedCount, winCount, lossCount
engine.orders	rows with broker , state , averageFillPrice
engine.positions	rows with broker , mark , trailing fields
engine.strategyStats	total 100 · eligible 2 · firstRelease 10
engine.universe	seeds · marketHours · preference · lastCycle

B Appendix B · The mode-lineage change (exact diffs that shipped in 4896a72)

Six surgical edits. Everything else in the pipeline is untouched — that is deliberate: mode lineage is a data property, so the change lives in the schema, the fill recorder, the projections, and the badges.

B.1 db/schema.ts — the column

```
/** Execution venue copied from the source ticket at fill time – PAPER | IBKR (mode lineage for reporting). */
broker: varchar("broker", { length: 24 }).notNull().default("PAPER"),
```

B.2 api/lib/ensure-schema.ts — the guarded migration

```
// Mode lineage on positions (PAPER | live broker) – copied from the source ticket at fill time
`ALTER TABLE positions ADD COLUMN broker VARCHAR(24) NOT NULL DEFAULT 'PAPER' AFTER sourceTicketId`,
```

B.3 api/engine/portfolio.ts — recordFill stamps the mode (new position)

```
sourceTicketId: ticket.ticketId,
broker: ticket.broker, // mode lineage: PAPER | live broker – reporting splits on this
```

B.4 api/engine/portfolio.ts — scale-out child keeps the parent's mode

```
sourceTicketId: ticket.ticketId,
broker: open.broker, // scale-out child keeps the ORIGINAL position's mode, not the sell ticket's
```

B.5 api/engine/portfolio.ts — mode-aware P&L split

```
export interface ModePnl {
  realizedTotal: number;
  unrealizedTotal: number;
  openCount: number;
  closedCount: number;
  winCount: number;
  lossCount: number;
}

function summarize(rows: PositionRow[]): ModePnl {
```

```
/* Mode-aware reporting: paper and live P&L are ALWAYS separable.
 * Combined totals above remain for back-compat; the byMode split is the
```

```
* authoritative view – a position's mode is fixed at fill time from its
* source ticket's broker (PAPER | live broker name). */
const byMode = {
  paper: summarize(rows.filter((r) => r.broker.toUpperCase() === "PAPER")),
  live: summarize(rows.filter((r) => r.broker.toUpperCase() !== "PAPER")),
};
```

B.6 Projections — broker on every row

```
avgEntry: Number(p.avgEntry),
broker: p.broker ?? "PAPER",
```

```
side: t.side,
broker: t.broker ?? "PAPER",
```

B.7 src/components/OrdersPanel.tsx — badges + mode cards

```
      {p.symbol} <Badge tone={p.broker === 'PAPER' ? 'sky' : 'teal'}>{p.broker ===
'PAPER' ? 'PAPER' : 'LIVE'}</Badge>
    </td>
```

```
      <Badge tone={o.broker === 'PAPER' ? 'sky' : 'teal'}>{o.broker === 'PAPER' ? 'PAPER'
: 'LIVE'}</Badge>
```

```
    /* Mode-aware P&L split – paper and live are always separable (positions carry broker lineage
from the source ticket) */
    {pnl?.byMode && (
      <div className="mb-5 grid gap-3 sm:grid-cols-2">
        <div className="rounded-xl border border-sky-200 bg-sky-50/50 p-3">
          <p className="text-[11px] font-bold uppercase tracking-wide text-sky-700">Paper mode (PAPER
broker)</p>
          <p className="mt-1 text-sm text-slate-700">
            Realized <b className={((pnl.byMode.paper.realizedTotal ?? 0) >= 0 ? 'text-teal-600' :
'text-red-600')}>{((pnl.byMode.paper.realizedTotal ?? 0) >= 0 ? '+' :
''}${(pnl.byMode.paper.realizedTotal ?? 0).toLocaleString()}</b>
            {' · '}Unrealized <b className={((pnl.byMode.paper.unrealizedTotal ?? 0) >= 0 ? 'text-
teal-600' : 'text-red-600')}>{((pnl.byMode.paper.unrealizedTotal ?? 0) >= 0 ? '+' :
''}${(pnl.byMode.paper.unrealizedTotal ?? 0).toLocaleString()}</b>
          </p>
          <p className="text-[11px] text-slate-500">{pnl.byMode.paper.openCount} open ·
{pnl.byMode.paper.closedCount} closed · {pnl.byMode.paper.winCount}W /
{pnl.byMode.paper.lossCount}L</p>
        </div>
        <div className="rounded-xl border border-teal-200 bg-teal-50/50 p-3">
          <p className="text-[11px] font-bold uppercase tracking-wide text-teal-700">Live mode (IBKR
/ real broker)</p>
```

```
Realized <b className={pnl.byMode.live.realizedTotal ?? 0} >= 0 ? 'text-teal-600' : 'text-red-600'}>
  {(pnl.byMode.live.realizedTotal ?? 0) >= 0 ? '+' : ''}${pnl.byMode.live.realizedTotal ??
0).toLocaleString()}</b>
    {' · '}Unrealized <b className={pnl.byMode.live.unrealizedTotal ?? 0} >= 0 ? 'text-teal-
600' : 'text-red-600'}>{(pnl.byMode.live.unrealizedTotal ?? 0) >= 0 ? '+' :
''}${pnl.byMode.live.unrealizedTotal ?? 0).toLocaleString()}</b>
    </p>
    <p className="text-[11px] text-slate-500">{pnl.byMode.live.openCount} open ·
{pnl.byMode.live.closedCount} closed · {pnl.byMode.live.winCount}W / {pnl.byMode.live.lossCount}L</p>
  </div>
</div>
  )}
```